

Python Basics

A concise reference covering the fundamentals of the Python programming language.

1. Introduction to Python

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability through significant indentation and a clean, simple syntax. Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming. It is widely used in web development, data science, automation, scripting, artificial intelligence, and scientific computing.

1.1 Key Features

- Easy to read and write, with simple, English-like syntax
- Interpreted language: code runs line by line without a separate compile step
- Dynamically typed: variable types are determined at runtime
- Large standard library and a huge ecosystem of third-party packages
- Cross-platform: runs on Windows, macOS, and Linux

1.2 Installing and Running Python

Python can be downloaded from python.org. Once installed, you can run the interactive interpreter by typing 'python' or 'python3' in a terminal, or run a script file with 'python script.py'.

```
$ python3 --version
Python 3.12.0
```

```
$ python3 hello.py
Hello, World!
```

2. Variables and Data Types

In Python, variables are created simply by assigning a value to a name; there is no need to declare a type in advance. Python automatically infers the type based on the assigned value.

```
x = 10          # int
y = 3.14        # float
name = "Alice"  # str
is_active = True # bool
items = [1, 2, 3] # list
```

2.1 Core Built-in Data Types

- **int** - whole numbers, e.g. 5, -12, 1000
- **float** - decimal numbers, e.g. 3.14, -0.5
- **str** - text strings, e.g. "hello"
- **bool** - True or False
- **list** - ordered, mutable collection, e.g. [1, 2, 3]
- **tuple** - ordered, immutable collection, e.g. (1, 2, 3)
- **dict** - key-value pairs, e.g. {"a": 1, "b": 2}
- **set** - unordered collection of unique items, e.g. {1, 2, 3}

2.2 Type Conversion

Python provides built-in functions to convert between types, such as `int()`, `float()`, `str()`, and `bool()`. This is commonly needed when reading user input, which is always returned as a string.

```
age = int("25")      # string to int
price = float("9.99") # string to float
label = str(42)       # int to string
```

3. Control Flow

Control flow statements determine the order in which code executes. Python uses indentation (typically 4 spaces) rather than curly braces to define code blocks.

3.1 Conditional Statements

```
age = 18

if age < 13:
    print("Child")
elif age < 20:
    print("Teenager")
else:
    print("Adult")
```

3.2 Loops

Python has two main looping constructs: 'for' loops, which iterate over a sequence, and 'while' loops, which repeat as long as a condition is true.

```
# for loop
for i in range(5):
    print(i)

# while loop
count = 0
while count < 5:
    print(count)
    count += 1
```

3.3 Loop Control Statements

- **break** - exits the loop immediately
- **continue** - skips to the next iteration
- **pass** - does nothing, used as a placeholder

4. Functions

Functions are reusable blocks of code defined with the 'def' keyword. They can accept parameters, provide default values, and return results using the 'return' statement.

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

message = greet("Alice")
print(message) # Hello, Alice!

message2 = greet("Bob", greeting="Hi")
print(message2) # Hi, Bob!
```

4.1 Lambda Functions

Lambda functions are small, anonymous, single-expression functions defined using the 'lambda' keyword. They are often used for short operations passed to other functions.

```
square = lambda x: x * x
print(square(5)) # 25
```

5. Data Structures in Practice

Lists and dictionaries are the most commonly used data structures in Python. Lists store ordered collections of items, while dictionaries map keys to values for fast lookup.

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")
print(fruits[0]) # apple

person = {"name": "Alice", "age": 30}
print(person["name"]) # Alice
person["email"] = "alice@example.com"
```

6. Object-Oriented Programming

Python supports object-oriented programming through classes. A class is a blueprint for creating objects that bundle data (attributes) and behavior (methods) together.

```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        return f"{self.name} says Woof!"

my_dog = Dog("Rex", "Labrador")
print(my_dog.bark()) # Rex says Woof!
```

7. Error Handling

Python uses try/except blocks to handle exceptions gracefully, preventing programs from crashing when errors occur at runtime.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
finally:
    print("Done")
```

8. Modules and Imports

Code can be organized into modules (individual .py files) and packages (folders of modules). The 'import' statement makes code from other modules available for use.

```
import math
print(math.sqrt(16)) # 4.0

from datetime import datetime
print(datetime.now())
```

This document provides a foundational overview of Python syntax and concepts, suitable as reference material for further study or as a knowledge base for retrieval-augmented applications.